

南開大學
Nankai University

计算机学院

《人工智能导论》课程实验报告

口罩佩戴检测

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

实验时间：2025 年 12 月 8 日 星期一

目录

1 问题重述 **2**

2 设计思想 **2**

2.1 总体框架：检测与分类 3

2.2 关键参数设置汇总 4

3 程序代码 **4**

3.1 train.py 4

3.2 main.py 9

4 测试结果 **11**

4.1 loss 与 acc 的可视化折线图 11

4.2 在线测试结果 12

5 总结 **12**

1 问题重述

背景介绍

今年一场席卷全球的新型冠状病毒给人们带来了沉重的生命财产的损失。有效防御这种传染病毒的方法就是积极佩戴口罩。我国对此也采取了严肃的措施，在公共场合要求人们必须佩戴口罩。在本次实验中，我们要建立一个目标检测的模型，可以识别图中的人是否佩戴了口罩。

实验要求

1. 建立深度学习模型，检测出图中的人是否佩戴了口罩，并将其尽可能调整到最佳状态。
2. 学习经典的模型 *MTCNN* 和 *MobileNet* 的结构。
3. 学习训练时的方法。

实验环境

可以使用基于 *Python* 的 *OpenCV*、*PIL* 库进行图像相关处理，使用 *Numpy* 库进行相关数值运算，使用 *PyTorch* 等深度学习框架训练模型等。

注意事项

Python 与 *Python Package* 的使用方式，可在右侧 *API* 文档中查阅。当右上角的“*Python 3*”长时间指示为运行中时，造成代码无法执行，可重新启动 *Kernel* 解决（左上角“*Kernel*” → “*Restart Kernel*”）。

参考资料

- 论文 *Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks*：https://kpzhang93.github.io/MTCNN_face_detection_alignment/
- *OpenCV*：https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_tutorials.html
- *PIL*：https://pillow.readthedocs.io/en/stable/
- *NumPy*：https://www.numpy.org/
- *Scikit-learn*：https://scikit-learn.org/
- *PyTorch*：https://pytorch.org/

2 设计思想

本实验的目标是对输入图像进行口罩佩戴情况的统计，即输出图像中的总人数 all_num 以及佩戴口罩的人数 $mask_num$。为更贴合该任务的输出形式，我们没有将问题简化为“整图二分类”，而是采用了两阶段流程：先进行**人脸检测**得到人脸区域，再对每个区域进行**口罩二分类**，最后完成计数统计。

2.1 总体框架：检测与分类

本方法由两部分组成：

- **人脸检测模块：**使用 MTCNN/FaceDetector 在复杂场景中定位所有人脸，解决多人、尺度变化、姿态差异等问题；
- **口罩分类模块：**使用轻量级网络 MobileNetV1 对裁剪后的人脸进行二分类（戴口罩/未戴口罩），即 $classes = 2$。

这种设计有利于检测与分类相互独立，便于调试与替换模块；更适合完成最终的统计目标。

为获得稳定可信的验证指标并提升模型泛化能力，本实验在训练脚本中主要从**数据划分与预处理**、**训练稳定性**以及**模型选择与保存策略**三个方面进行优化。

(1) 数据划分与预处理：保证评估可信

首先，为避免训练集与验证集类别比例差异带来的评估偏置，采用 `stratified_split` 进行分层划分，使两部分数据分布尽可能一致。其次，训练集与验证集的变换严格分离：训练集加入 `RandomHorizontalFlip` 等轻度增强以提升泛化；验证集仅保留 `Resize + Normalize` 等确定性操作，从而保证 val_acc 的可比性与稳定性。考虑到数据集中可能存在灰度图，预处理阶段统一执行 `convert("RGB")`，确保输入通道一致。

(2) 训练稳定性：优化器、调度器与早停

模型采用 `MobileNetV1(classes=2)` 完成口罩二分类任务。优化器选用 `AdamW`，以获得更平滑的收敛过程并提升泛化能力。学习率调度使用 `ReduceLROnPlateau(mode="max")` 并以验证集准确率为监控指标，当性能进入平台期时按 `factor=0.5` 自动降低学习率（`patience=3`，`min_lr=1e-6`），从而利于后期进一步细化收敛。

为减少过拟合与无效迭代，引入 Early Stopping：当验证集准确率连续若干轮不再提升时提前终止训练（本实验设置为 `early_stop_patience=8`）。此外，在检测到 GPU 的情况下启用 AMP 混合精度训练，以加速训练并降低显存占用。若类别分布不均衡，则统计训练集类别数量得到 `class_weights` 并加入交叉熵损失进行补偿；同时在环境支持时启用 `label_smoothing=0.1` 以缓解过拟合并提升泛化表现。

(3) 最优模型保存：以验证集表现为准

训练过程中不以最后一轮为准，而是以验证集指标 val_acc 为标准保存最优权重：当 val_acc 提升时立即更新并保存到 `./results/temp.pth`；训练结束后再次加载最优权重并保存，保证最终推理阶段使用的模型确为验证集性能最佳的版本，同时获得 loss 和 acc 随 Epoch 变化的可视化折线图。

2.2 关键参数设置汇总

本实验最终采用的关键训练配置如下：

$epochs = 40$

$model = MobileNetV1(classes = 2)$

$optimizer = \texttt{AdamW}(lr = 3 \times 10^{-4},\ weight_decay = 10^{-4})$

$scheduler = \texttt{ReduceLROnPlateau}(mode = \texttt{max},\ factor = 0.5,\ patience = 3,\ min_lr = 10^{-6})$

$early_stop_patience = 8$

$save_path = \texttt{./results/temp.pth}$

通过上述设计，训练过程更稳定、验证评估更可信，同时能够自动保存验证集最优模型，为最终推理统计任务提供可靠支撑。

3 程序代码

3.1 train.py

```python
import warnings
warnings.filterwarnings("ignore")

import os
import random
import copy
import numpy as np
import matplotlib.pyplot as plt
from tqdm.auto import tqdm

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision.datasets import ImageFolder
import torchvision.transforms as T

from torch_py.MobileNetV1 import MobileNetV1


# ---------------------------
# 1) 复现性：固定随机种子
# ---------------------------
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False

# ---------------------------
# 2) 分层划分
```

```python
# --------------------------
def stratified_split(targets, test_split=0.1, seed=42):
    targets = np.asarray(targets)
    idx = np.arange(len(targets))
    rng = np.random.RandomState(seed)

    train_idx_parts = []
    val_idx_parts = []

    classes = np.unique(targets)
    for c in classes:
        c_idx = idx[targets == c]
        rng.shuffle(c_idx)

        n_val = int(len(c_idx) * test_split)
        val_idx_parts.append(c_idx[:n_val])
        train_idx_parts.append(c_idx[n_val:])

    train_idx = np.concatenate(train_idx_parts)
    val_idx = np.concatenate(val_idx_parts)

    rng.shuffle(train_idx)
    rng.shuffle(val_idx)

    return train_idx.tolist(), val_idx.tolist()


# --------------------------
# 3) 数据处理
# --------------------------
def processing_data(data_path, height=160, width=160, batch_size=32, test_split=0.1,
                    seed=42, num_workers=0):
    # 先创建一个不带 transform 的 dataset 用来拿 targets 做分层划分
    base_ds = ImageFolder(data_path)
    targets = base_ds.targets

    train_idx, val_idx = stratified_split(targets, test_split=test_split, seed=seed)

    to_rgb = T.Lambda(lambda img: img.convert("RGB"))

    # 训练集增强
    train_tfms = T.Compose([
        to_rgb,
        T.Resize((height, width)),
        T.RandomHorizontalFlip(p=0.5),
        T.RandomRotation(15),
        T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.02),
        T.ToTensor(),
        T.Normalize(mean=[0.5, 0.5, 0.5],
                    std=[0.5, 0.5, 0.5]),
    ])

    # 验证集
    val_tfms = T.Compose([
        to_rgb,
        T.Resize((height, width)),
        T.ToTensor(),
        T.Normalize(mean=[0.5, 0.5, 0.5],
                    std=[0.5, 0.5, 0.5]),
```

```
    ])

    # 构建两个 dataset (各自 transform 不同)
    train_ds_full = ImageFolder(data_path, transform=train_tfms)
    val_ds_full = ImageFolder(data_path, transform=val_tfms)

    train_dataset = Subset(train_ds_full, train_idx)
    val_dataset = Subset(val_ds_full, val_idx)

    train_targets = np.array(targets)[train_idx]
    class_count = np.bincount(train_targets)
    class_count = np.maximum(class_count, 1)
    class_weights = class_count.sum() / (len(class_count) * class_count)
    class_weights = torch.tensor(class_weights, dtype=torch.float32)

    pin_memory = torch.cuda.is_available()
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
                              num_workers=num_workers, pin_memory=pin_memory)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False,
                            num_workers=num_workers, pin_memory=pin_memory)

    print("Class mapping:", train_ds_full.class_to_idx)
    print("Train size:", len(train_dataset), " Val size:", len(val_dataset))
    print("Train class_count:", class_count, " class_weights:", class_weights.tolist())

    return train_loader, val_loader, class_weights

# ---------------------------
# 4) 评估函数
# ---------------------------
@torch.no_grad()
def evaluate(model, loader, criterion, device, use_amp=False):
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0

    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        if use_amp:
            with torch.cuda.amp.autocast():
                logits = model(x)
                loss = criterion(logits, y)
        else:
            logits = model(x)
            loss = criterion(logits, y)

        total_loss += loss.item() * x.size(0)
        pred = logits.argmax(dim=1)
        correct += (pred == y).sum().item()
        total += y.size(0)

    return total_loss / max(total, 1), correct / max(total, 1)


# ---------------------------
# 5) 训练1个epoch
```

```python
# --------------------------
def train_one_epoch(model, loader, optimizer, criterion, device, scaler=None, use_amp=False):
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    for x, y in tqdm(loader, desc="train", leave=False):
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)

        if use_amp:
            with torch.cuda.amp.autocast():
                logits = model(x)
                loss = criterion(logits, y)
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
        else:
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward()
            optimizer.step()

        total_loss += loss.item() * x.size(0)
        pred = logits.argmax(dim=1)
        correct += (pred == y).sum().item()
        total += y.size(0)

    return total_loss / max(total, 1), correct / max(total, 1)

# --------------------------
# 6) 主训练流程
# --------------------------
def main():
    set_seed(42)

    # ===== 路径 =====
    data_path = "./datasets/5f680a696ec9b83bb0037081-momodel/data/image"
    os.makedirs("./results", exist_ok=True)
    save_path = "./results/temp.pth"
    fig_path = "./results/temp_metrics.png"

    # ===== 设备 =====
    device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
    print("Device:", device)

    # ===== 数据 =====
    train_loader, val_loader, class_weights = processing_data(
        data_path=data_path,
        height=160, width=160,
        batch_size=32,
        test_split=0.1,
        seed=42,
        num_workers=0
    )

```

```
    # ===== 模型 =====
    model = MobileNetV1(classes=2).to(device)

    # ===== 优化器=====
    optimizer = optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)

    # ===== 学习率调度=====
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="max", factor=0.5, patience=3, verbose=True, min_lr=1e-6
    )

    # ===== Loss =====
    try:
        criterion = nn.CrossEntropyLoss(weight=class_weights.to(device), label_smoothing=0.1)
        print("Use CrossEntropyLoss(label_smoothing=0.1)")
    except TypeError:
        criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
        print("Use CrossEntropyLoss(with class weight), label_smoothing not supported in this torch.")

    # ===== AMP =====
    use_amp = torch.cuda.is_available()
    scaler = torch.cuda.amp.GradScaler(enabled=use_amp)

    # ===== 训练配置 =====
    epochs = 40
    early_stop_patience = 8

    # ===== 记录 =====
    hist = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": [], "lr": []}

    # ===== best 保存 =====
    best_val_acc = 0.0
    best_state = copy.deepcopy(model.state_dict())
    wait = 0

    for epoch in range(1, epochs + 1):
        train_loss, train_acc = train_one_epoch(
            model, train_loader, optimizer, criterion, device, scaler=scaler, use_amp=use_amp
        )
        val_loss, val_acc = evaluate(
            model, val_loader, criterion, device, use_amp=use_amp
        )

        scheduler.step(val_acc)
        lr = optimizer.param_groups[0]["lr"]

        hist["train_loss"].append(train_loss)
        hist["train_acc"].append(train_acc)
        hist["val_loss"].append(val_loss)
        hist["val_acc"].append(val_acc)
        hist["lr"].append(lr)

        print(f"Epoch [{epoch:02d}/{epochs}] "
              f"lr={lr:.2e} | "
              f"train_loss={train_loss:.4f} train_acc={train_acc:.4f} | "
              f"val_loss={val_loss:.4f} val_acc={val_acc:.4f}")

        # 保存 best
        if val_acc > best_val_acc:
```

```
            best_val_acc = val_acc
            best_state = copy.deepcopy(model.state_dict())
            torch.save(best_state, save_path)
            wait = 0
            print(f"   Save best -> {save_path} (best_val_acc={best_val_acc:.4f})")
        else:
            wait += 1
            if wait >= early_stop_patience:
                print("   Early stopping triggered.")
                break

    # 训练结束
    model.load_state_dict(best_state)
    torch.save(model.state_dict(), save_path)
    print(f"Finish Training. Best val_acc={best_val_acc:.4f}, saved to {save_path}")

    # ===== 画图并保存 =====
    fig = plt.figure(figsize=(10, 4))
    ax1 = fig.add_subplot(1, 2, 1)
    ax1.plot(hist["train_loss"], label="train_loss")
    ax1.plot(hist["val_loss"], label="val_loss")
    ax1.set_title("Loss")
    ax1.set_xlabel("Epoch")
    ax1.set_ylabel("Loss")
    ax1.legend()

    ax2 = fig.add_subplot(1, 2, 2)
    ax2.plot(hist["train_acc"], label="train_acc")
    ax2.plot(hist["val_acc"], label="val_acc")
    ax2.set_title("Accuracy")
    ax2.set_xlabel("Epoch")
    ax2.set_ylabel("Acc")
    ax2.legend()

    fig.tight_layout()
    plt.savefig(fig_path, dpi=200)
    plt.show()
    print(f"Metrics figure saved to {fig_path}")


if __name__ == "__main__":
    main()
```

Listing 1: train.py

3.2 main.py

```

import warnings
# 忽视警告
warnings.filterwarnings('ignore')

import cv2
from PIL import Image
import numpy as np
import copy
import matplotlib.pyplot as plt
```

```python
from tqdm.auto import tqdm
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision.datasets import ImageFolder
import torchvision.transforms as T
from torch.utils.data import DataLoader

from torch_py.Utils import plot_image
from torch_py.MTCNN.detector import FaceDetector
from torch_py.MobileNetV1 import MobileNetV1
from torch_py.FaceRec import Recognition

from torch_py.Utils import plot_image
from torch_py.MTCNN.detector import FaceDetector
from torch_py.MobileNetV1 import MobileNetV1
from torch_py.FaceRec import Recognition
from torch_py.FaceRec import Recognition
from PIL import Image
import cv2

# --------------------------- 请加载您最满意的模型 ---------------------------
# 加载模型(请加载你认为的最佳模型)
# 加载模型,加载请注意 model_path 是相对路径, 与当前文件同级。
# 如果你的模型是在 results 文件夹下的 dnn.h5 模型, 则 model_path = 'results/temp.pth'
model_path = './results/temp.pth'
# -----------------------------------------------------------------------------

def predict(img):
    """
    加载模型和模型预测
    :param img: cv2.imread 图像
    :return: 预测的图片中的总人数、其中佩戴口罩的人数
    """
    # --------------------------- 实现模型预测部分的代码 ---------------------------
    # 将 cv2.imread 图像转化为 PIL.Image 图像, 用来兼容测试输入的 cv2 读取的图像 (勿删!!!)
    # cv2.imread 读取图像的类型是 numpy.ndarray
    # PIL.Image.open 读取图像的类型是 PIL.JpegImagePlugin.JpegImageFile
    if isinstance(img, np.ndarray):
        # 转化为 PIL.JpegImagePlugin.JpegImageFile 类型
        img = Image.fromarray(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))

    recognize = Recognition(model_path)
    img, all_num, mask_num = recognize.mask_recognize(img)
    # -----------------------------------------------------------------------------
    return all_num,mask_num
```

Listing 2: train.py

4 测试结果

4.1 loss 与 acc 的可视化折线图

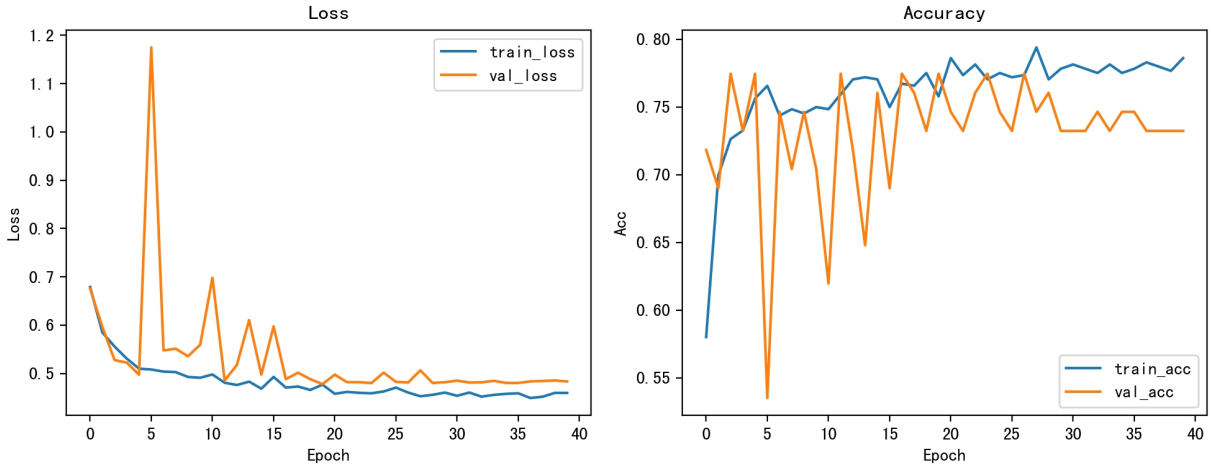


图 4.1.1: loss 与 acc 的可视化折线图

我们通过观察 train$_l$oss val_loss $train_a$cc val_acc (*loss* 40 *epoch* *acc* *epoch* 4

4.2 在线测试结果



图 4.2.1: 模型测试结果

经在线测试，在五张图片上测试模型结果得分为 100。

5 总结

通过本次实验，我不仅完成了“统计图像中总人数与戴口罩人数”的任务，更重要的是系统体会到了**训练调参与训练流程设计**在深度学习项目中的核心作用。实验初期模型效果并不理想，验证集指标波动明显，推理结果也不够稳定。为此，我围绕训练脚本 `train.py` 进行了多轮迭代式优化：从数据划分方式、数据增强策略，到优化器与学习率调度器的配置，再到早停与最优模型保存逻辑，每一次调整都对应着一次“定位问题—提出假设—修改参数—观察曲线—再迭代”的过程。随着训练曲线逐步变得平滑、验证准确率更稳定，最终得到可用于推理端 `main.py` 的最优权重文件 `./results/temp.pth`，并在测试中取得了更理想的效果。

在调参过程中，我总结出几条非常关键的经验：当验证集准确率起伏较大时，往往不是模型本身的问题，而可能来自验证集的随机增强或划分偏置；因此本实验采用了 `stratified_split` 的分层划分，并将训练集与验证集的 transforms 严格分开，使得 val_acc 更可信、更具可比性。与此同时，针

对训练后期进入平台期的问题，我使用 `ReduceLROnPlateau(mode="max")` 以验证准确率为依据自动降低学习率（`factor=0.5, patience=3`），配合 `AdamW(lr=3e-4, weight_decay=1e-4)` 获得更稳定的收敛；当连续多轮无提升时，再通过 Early Stopping（`early_stop_patience=8`）及时终止训练，避免无效迭代与过拟合。此外，为应对类别不平衡带来的偏置，本实验计算 `class_weights` 并加入交叉熵损失中；在环境支持时还启用 `label_smoothing=0.1`，进一步增强泛化能力。整个过程让我更清晰地理解了：什么时候需要调参（指标波动/不过拟合/过拟合/收敛停滞）、往哪个方向调（数据、优化、正则、训练策略）、以及如何用曲线与验证集表现去验证调整是否有效。